

第二章 进程管理

同济大学计算机学院 国豪、拔尖班《操作系统》课程讲义 2025 年 10 月 1 日 邓蓉

第一节、进程的表示

一、进程图像 (Process Image)

1、进程是应用程序的一个执行实例。每个进程执行 2 个程序：1 个应用程序和 1 个操作系统内核。

```
#include <stdio.h>
int version=1;
main1()
{
    int a,b,result;
    a = 1;
    b = 2;
    result = sum(a,b);
    printf("result=%d\n",result);
}
int sum(var1, var2)
{
    int count;
    version = 2;
    count = var1 + var2;
    return(count);
}
```

图 1、Unix V6++系统中的 ShowStack 进程

2、用户态、核心态；Unix 进程的用户栈和核心栈



每个进程有两种执行状态 (Unix系统)

- 用户态，执行应用程序，访问应用程序的所有逻辑段。
- 核心态，执行操作系统内核，访问内核代码 和 数据。

用户栈 和 核心栈 (Unix进程)

- 执行应用程序时，使用用户栈。
- 执行系统调用时，使用核心栈。

图 1, ShowStack 进程，用户态运行时执行应用程序 ShowStack，核心态运行时执行内核 Kernel.exe

3、进程图像包含，进程控制块 (Process Control Block, 简称 PCB)，应用程序，操作系统内核。 是的，包括应用程序，操作系统内核的全部代码、数据。



图 2、进程图像

3.1 PCB

PCB 是登记进程执行状态的内核数据结构，每个进程 1 个，创建进程时分配，进程终止时回收。Unix V6++的 PCB 包括 2 个数据结构，Process 和 User。

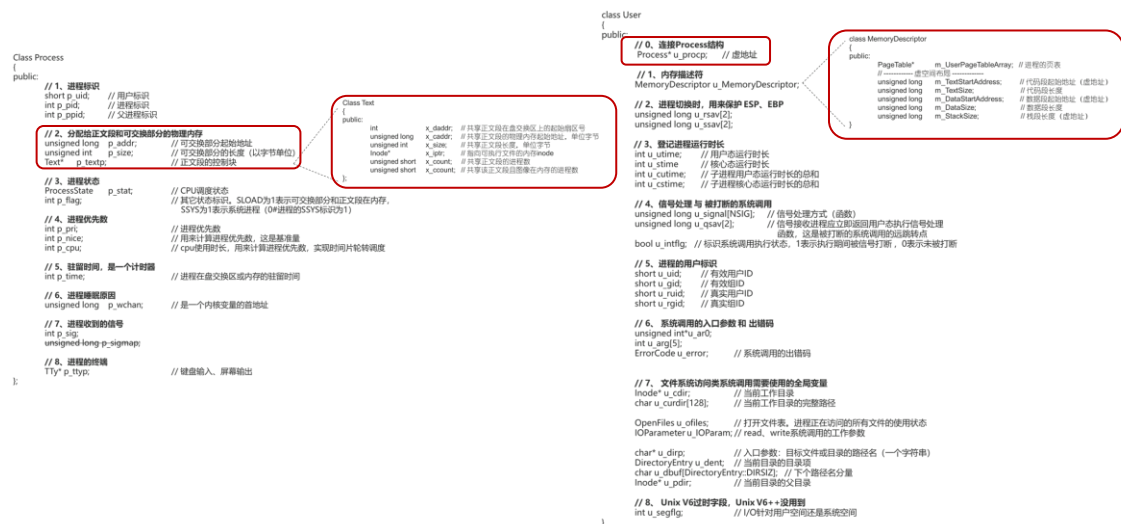


图 3、Unix V6++的进程控制块
精修了字段顺序，将各模块主理的字段收集在了一起，逻辑清晰很多。

注意 1，所有字段描述这个进程的状态。

注意 2，这个结构跑不起来。为什么呢？要再学好一会。彼时，小修内核，让我们的系统跑出来这个 PCB。先动手的同学，切记，clean、all、run，让你对数据结构的修改完全生效。

注意 3，Unix V6++进程的 PCB，好庞大的数据结构。emm…… Windows 进程、Linux 进程的 PCB 更大，每个超过 1000 字节。

理解全部字段需要学更多。先浏览代码和注释。关注红框中与进程图像相关的信息，讲义马上会用到。

3.2 逻辑段

编译器（比如，gcc）编译程序时，将属于相同属性的所有元素（指代码和数据）收集在一起，形成一个个的访问属性不同的逻辑段。应用程序是这样，内核亦是如此。如图 2，细节请参考《gcc 的编译链接过程》。



进程图像 (用户空间 & 核心空间)

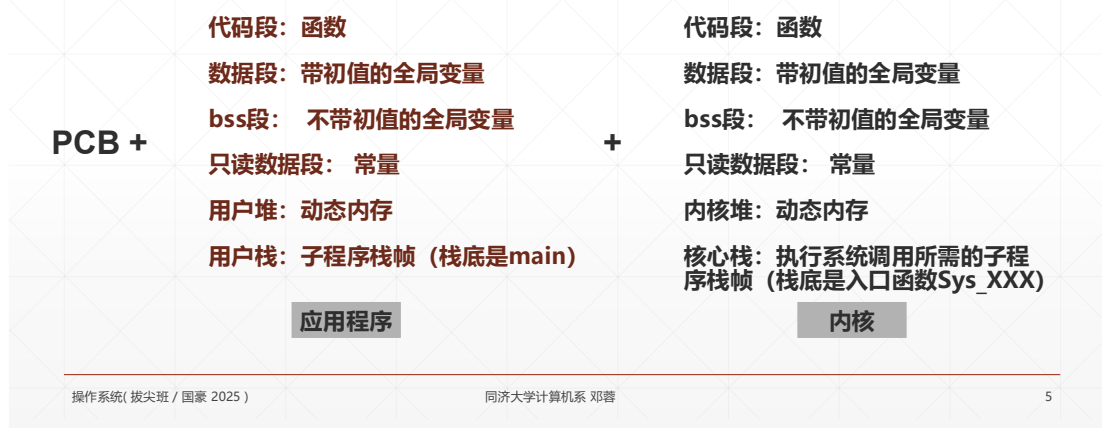


图 4

红色是进程的用户空间，包含应用程序所有逻辑段。黑色是进程的核心空间，包含内核所有逻辑段。

进程用户态运行时，使用用户栈，访问用户空间（中的代码、常量、变量），不能碰内核；因为应用程序的入口是 main，所以用户栈底是 main 栈帧。

进程核心态运行时，使用核心栈，执行内核空间中的代码。既可以访问核心空间，也可以访问用户空间（访问用户空间，内核可以为应用程序提供数据、获取系统调用所需参数）。进程执行内核的起点是应用程序执行系统调用，所以核心栈底是系统调用入口函数（Sys_XXX）的栈帧。[注：激活内核的还可能是中断和异常，所以栈底帧也可能是中断或异常的入口函数的栈帧，后面会认真说，是重点。先巩固已经知道的]



进程图像 (进程私有 & 所有进程共用)

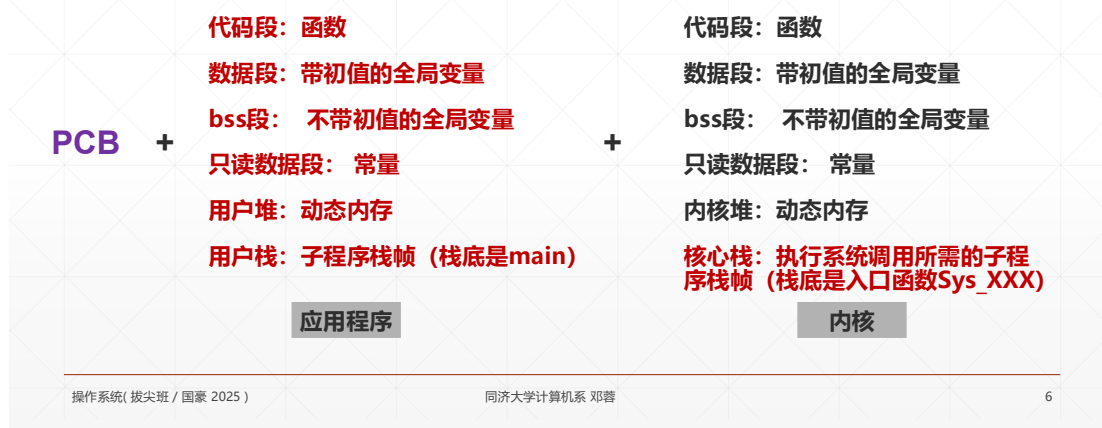


图 5

红色是进程私有的，其它进程不可以碰（核心态运行时也没权限）。

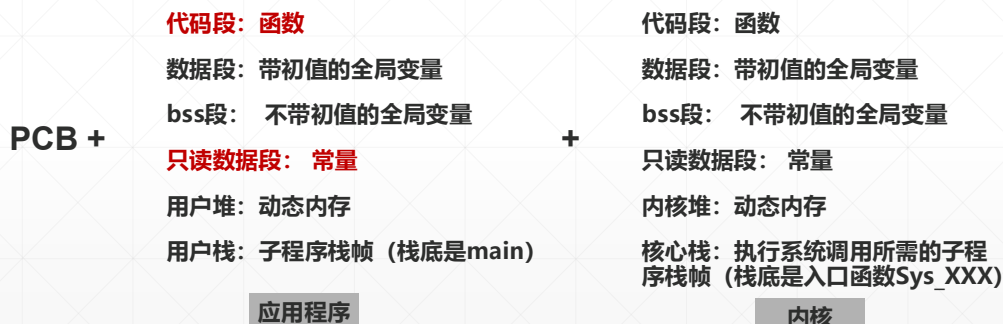
黑色，所有进程共用一个 copy。对的，在计算机系统中，内核只有一个副本，所有进程共用：它们用相同的虚地址访问同一个内核函数或同名内核变量（虚空间，一个副本供所有进程使用），位于物理内存的同一个位置。

PCB: Process 结构，其它进程可以碰，User 结构不行。进程访问其它进程的 Process 结构，是为了……向它发信号，维护其调度状态……很快我们会领会，放在这里，我们继续。



进程图像（执行同一个应用程序的所有进程共享正文段）

因为程序执行时，不会更改代码 和 只读数据



操作系统(拔尖班 / 国家 2025)

同济大学计算机系 邓蓉

7

图 6

应用程序执行时，不会改变代码和只读数据。将这 2 部分合并，共享正文段。

执行同一个应用程序的所有进程共享正文段，可以节省进程图像的内存消耗。

```
Class Text
{
public:
    int          x_daddr; // 共享正文段在盘交换区上的起始扇区号
    unsigned long x_caddr; // 共享正文段的物理内存起始地址，单位字节
    unsigned int  x_size;  // 共享正文段长度，单位字节
    Inode*        x_iptr;  // 指向可执行文件（应用程序）的内存inode
    unsigned short x_count; // 共享正文段的进程数
    unsigned short x_ccount; // 共享该正文段且图像在内存的进程数
};
```

Text 是共享正文段的控制块，登记正文段的使用情况，看代码注释。

总结：Unix V6++进程图像，2 部分



Unix V6++ 进程图像 (私有)

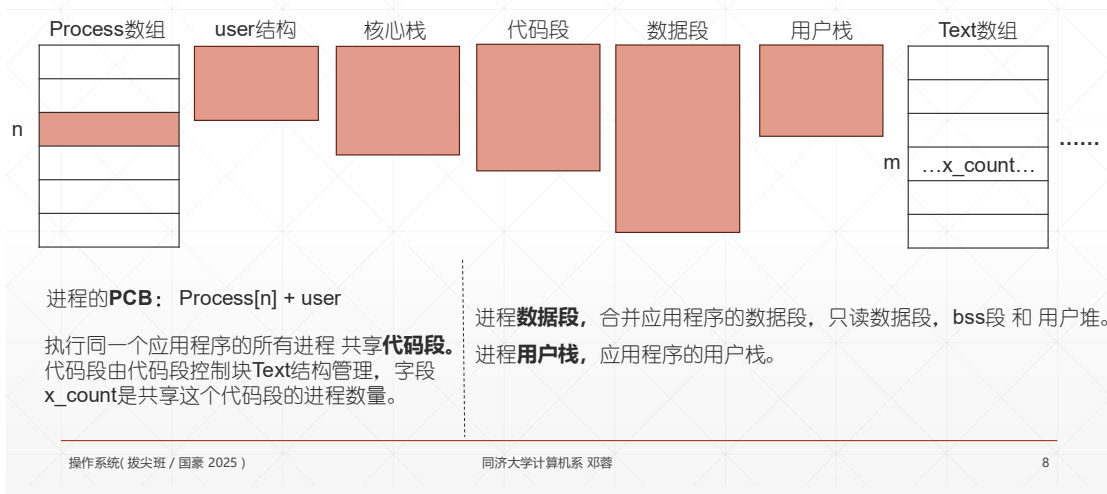


图 7

橘红色的是进程的私有图像。



Unix V6++ 进程图像 (所有进程共享)

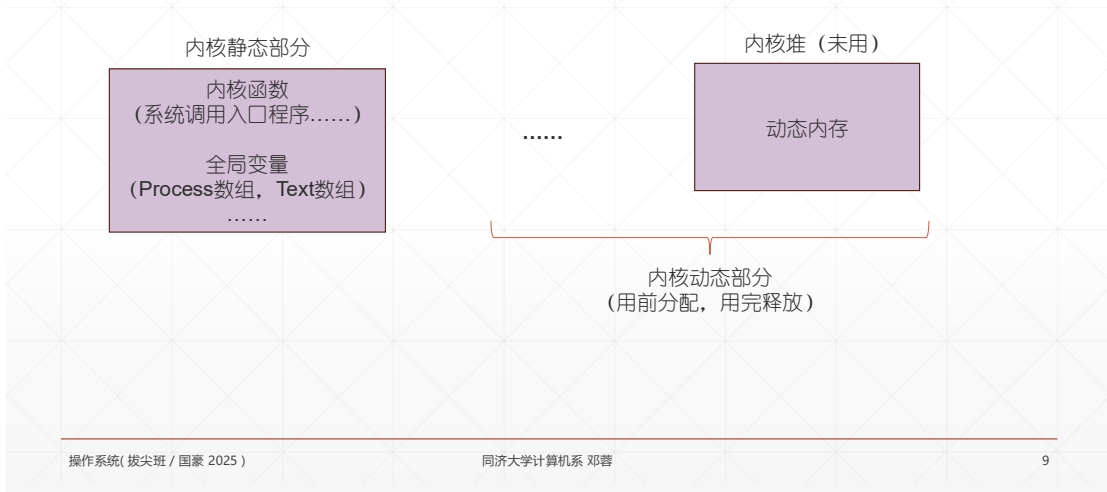


图 8

紫色的和其它进程共享。内核堆, 后续, 学有余力的同学, 我们想办法用好它。

二、虚空间 和 物理空间中的进程图像

1、虚空间中的进程图像

1.1 进程的虚空间

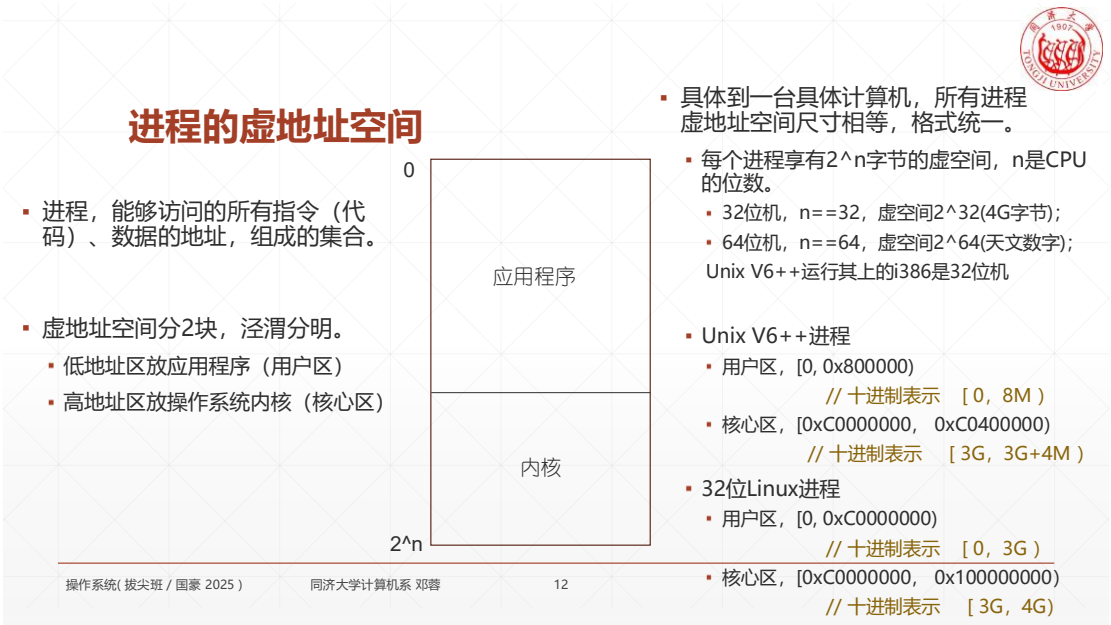


图 9

1.2 Unix V6++进程的虚空间

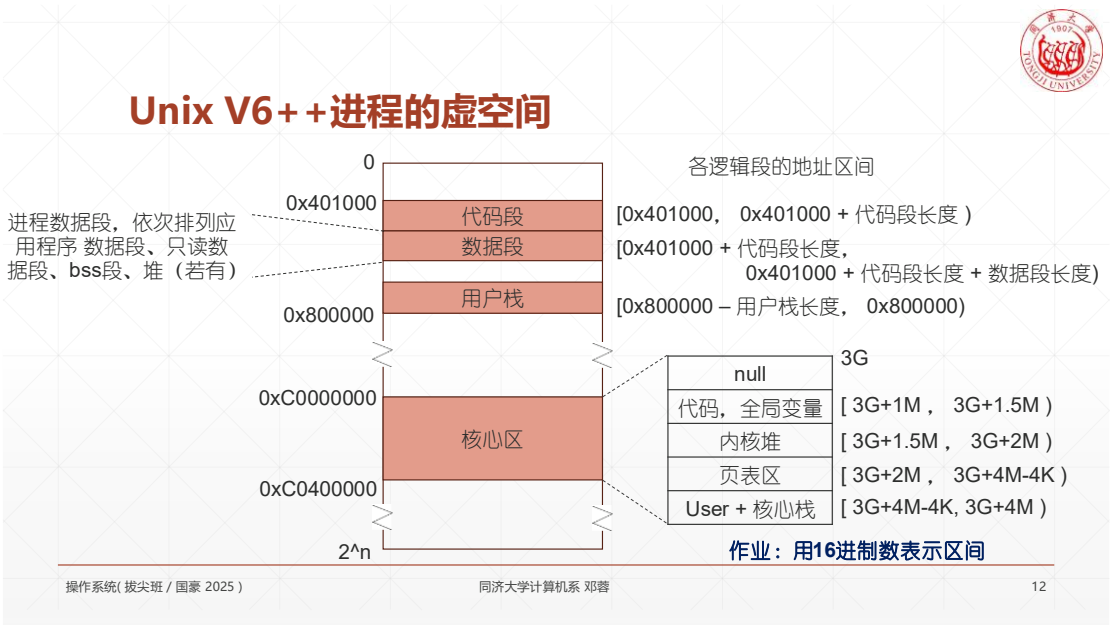


图 10

应用程序的逻辑段 4096 字节对齐（4K 对齐）。

将数据段、只读数据段、bss 和堆合并成一个大的数据段。

虚空间中[0, 0x800000)是**用户区**，共计 8M 字节。先排代码段，随后紧贴存放数据段。堆栈段在用户空间的最高端，栈底 0x800000。代码段的首地址 imageBegin 是 0x401000，登记在可执行文件程序头。

虚空间中[0xC0000000, 0xC0400000)是**核心区**，共计 4M 字节。依然先排代码段，随后紧贴存放数据段。User 结构地址 0xC03ff000；核心栈在用户空间的最高端，栈底 0xC0400000。是的，进程运行时，用虚地址 0xC03ff000 访问自己的 user 结构。它看不见其它进程的 user 结构和核心栈。

PPT 13~16，略。

1.3 物理空间中的进程图像

物理空间存放代码和数据。访问代码或数据时，CPU 需要给出物理地址。物理空间的存储介质是内存和磁盘上的盘交换区。因为专门用来存放进程图像，又称可执行存储器。

内存是主存储器，能够容纳全部进程图像时，没必要使用盘交换区，后者是空的；内存紧张时，系统会将一部分图像放在盘交换区上，这些盘交换区上的进程不可执行，系统资源供内存中的进程使用；每过一段时间，对换内存和盘交换区上的进程，这样所有进程可以轮流使用内存资源，都有运行的机会。

Unix V6++，内存分成 2 部分，系统区存放内核，用户区存放正文段和可交换部分。多道系统并发很多进程，所以（1）用户区有好多正文段，好多可交换部分（2）设置 0# 进程，执行调度和系统维护任务。0# 进程是内核线程，只执行内核函数，不跑应用程序；它的可交换部分只有 4096 字节 PPDA，没有代码段，数据段和用户栈。

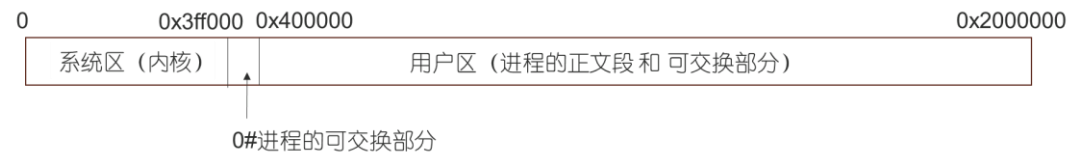


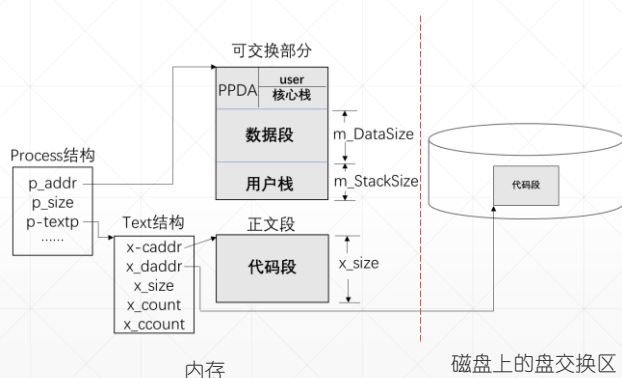
图 10、Unix V6++ 的内存。

物理地址 [0, 0x400000)，系统区 4M 字节；
[0x400000, 0x2000000)，用户区 32M 字节

Unix V6++ 进程图像，分 3 块，Process 结构，共享正文段 和 可交换部分。



物理空间中的进程图像



- **Process结构**存放在Process数组。
- **共享正文段**，包括代码段和只读数据段 (Unix V6++ buggy, 把只读数据段放在了可交换区)，可与执行同一应用程序的其它进程共享。这是为了节省进程图像的内存消耗。
- **可交换部分**，包括PPDA区，数据段和用户栈。PPDA (Per Process Data Area) 区，4096字节，低地址端是user结构，高地址端是进程的核栈。可交换部分禁止其它进程访问。

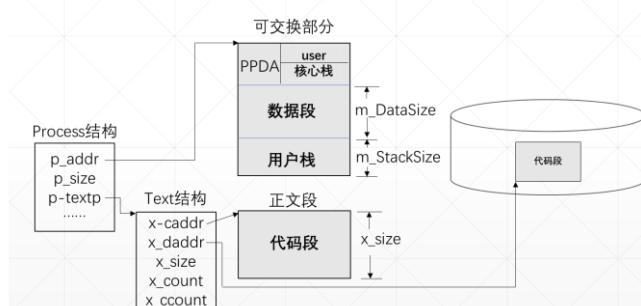
p_addr，可交换部分在物理空间的起始地址。内存中的进程，这个字段是起始**物理地址**（单位，字节）；盘交换区上的进程，是起始扇区号（一个扇区 512 字节）。

p_size，可交换部分长度（单位，字节）。

p_textp（虚地址），指向正文段控制结构 Text。其中

- x_caddr，正文段复本在内存中的起始地址。盘交换区上的进程，x_caddr 为 null。
- x_daddr，正文段复本在磁盘上的起始地址

上图中，m_DataSize 和 m_StackSize 是数据段和堆栈段的长度，登记在 User 结构中，是内存描述符 u_MemoryDescriptor 中的字段。除 0#进程外，每个进程有一个内存描述符，登记代码段、数据段和用户栈在虚空间中的起始地址和长度，是极其重要的内存数据结构。



- Process结构（进程基本控制块）常驻内存。
- 共享正文段，内存、磁盘各存一个副本。
- 内存紧张时，进程的可交换部分可暂存于磁盘，以释放内存空间。必要时，还会释放正文段占据的内存。磁盘上，用来存放代码段和可交换部分的区域，叫做盘交换区。
- 代码段、可交换部分全部在内存，进程可以执行（user在内存中）。否则，进程图像在盘交换区上，进程不可执行。

看一些细节：

- 回到图 10，虚空间中数据段和用户栈之间没有代码和数据，是空的，没必要分配物理空间。所以，物理空间（可交换部分）中，用户栈与数据段紧贴，并没有为 null 的虚空间分配物理内存。节约内存。
- 核心空间的最后一页是进程的 User 结构和核心栈（PPDA 区）。这 1 页放在可交换部分，第 1 个物理页框，与数据段、用户栈贴一起。User 结构中的信息，比如执行系统调用所需的全局变量，进程运行时才需要。核心栈中的栈帧，也是运行时才需要。把它们和数据段、用户栈放一起。。。这是进程运行时需要访问的私有数据，全部!!! PPDA，名字起得很棒很棒，随着课程进展，慢慢体会。
- 共享正文段有 2 个副本，内存一个，盘交换区一个。这个设计可以提高系统运行效率，最主要滴，执行换出操作时，不需要将正文段写回磁盘（盘交换区）。这个比较不重要，我们还是先关注内存。



进程图像在物理内存中的位置



图：内存中的 ShowStack 进程

Process 结构存放在系统区，Process 数组。本例，分配给 ShowStack 进程的是 Process[n]元素；

Text 结构存放在系统区，Text 数组。本例，分配给 ShowStack 进程的是 Text[m]元素；

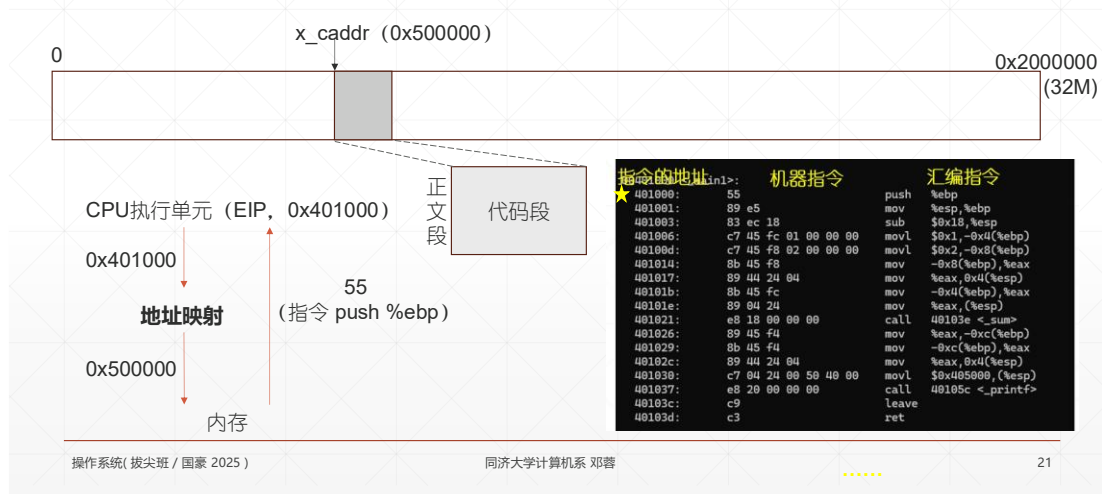
正文段在用户区。本例，分配给 ShowStack 程序的是起始地址是 0x500000 的连续内存块。里面存放着比如说，ShowStack 程序，全部指令。图示，黑底白色的块展示的是该程序 main 函数的 15 条指令；

可交换部分也在用户区。本例，起始地址是 0x700000。

1.4 地址映射



地址映射，取指 push %ebp



我们用一个具体的例子，说清楚 CPU 为什么需要设置地址映射元件。

场景：ShowStack 进程取指第一条指令 push %ebp

首先，观察 objdump 的输出，该指令虚地址 0x401000，对应的机器码是 0x55。这意味着：

- CPU 执行单元给出虚地址 0x401000，可以获得 push %ebp 指令。

另一方面，这条指令是代码段的第一条指令，存放在物理地址是 0x500000 的内存单元（本次执行系统为代码段分配的内存，首地址是 x_caddr == 0x500000）。看图。这意味着：

- CPU 想要从内存获得指令 push %ebp 的机器码 0x55，就需要向内存提供物理地址 0x401000。

所以，在 CPU 执行单元和内存单元之间，一定要有一个以硬件为中心的**地址映射机制**，将**虚地址（逻辑地址）** 0x401000 **翻译成** 指令在内存中的**物理地址** 0x500000。

Unix V6++的地址映射机制

地址映射由 CPU 硬件和操作系统内核共同完成。具体而言，

- 内核为每个进程设置一组数据结构，登记该进程的地址映射关系。
- 运行时，CPU 硬件使用操作系统为该进程提供的数据结构，将执行单元给出的逻辑地址（VA，Virtual Address）翻译成物理地址（PA，Physical Address）。执行地址映射的是内存管理单元（MMU，memory management unit）。这个重要元件，地位不亚于负责执行指令的 CPU 执行单元。

虚空间分页

具体到 Unix V6++ 系统。

- i386 MMU 硬件以页为粒度管理进程图像。页，又叫逻辑页，尺寸是 4096 字节[注 1]。
- 虚空间分页。Unix V6++ 进程为例，赘述之：
 - 虚地址 [0, 0x1000), 页号 0 ……
 - 虚地址 [0x1000, 0x2000), 页号 1, 为空、没有代码、数据
 - ……为空、没有代码、数据……
 - 虚地址 [0x401000, 0x402000), 页号 0x401 (代码段的第 1 页)
 - 代码段其它页
 - **数据段**的所有逻辑页
 - ……为空、没有代码、数据……
 - 虚地址 [0x7ff000, 0x800000), 页号 0x7ff (用户栈的页，位于栈底)。用户空间至此结束。
 - ……为空、没有代码、数据……
 - 虚地址 [0xC0100000, 0xC0101000), 页号 0xC0100 (内核代码，第 1 页)。核心空间从 0xC0000000 开始，不纠结细节。
 - ……内核代码、数据、堆 和 页表区
 - 虚地址 [0xC03ff000, 0xC0400000), 页号 0xC03ff (内核最后一页，进程的 PPDA)。核心空间至此结束。

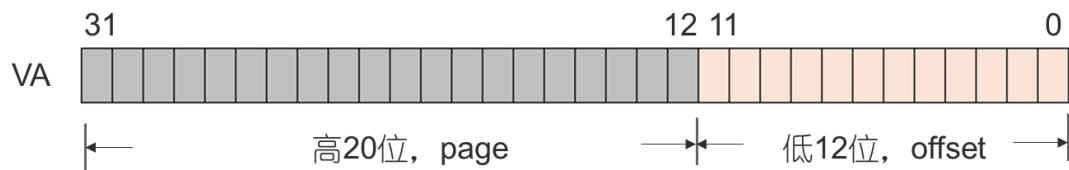
代码/数据的页号 和 页内偏移量

虚地址为 VA 的代码或数据，

页号 $page = VA / 4096$ 的商，页内偏移量 $offset = VA / 4096$ 的余数

CPU 不擅长执行除法操作，会超级慢。

这里，除数是 2^n , $n=12$ ，求取 page、offset，MMU 的做法耗时 ≈ 0 。



32 位逻辑地址，取高位 20bit，页号；低位 12bit，页内偏移量。

物理内存分页

按页的粒度，将物理内存分成一个一个的物理页框。每个物理页框可以容纳一页。再赘述之：

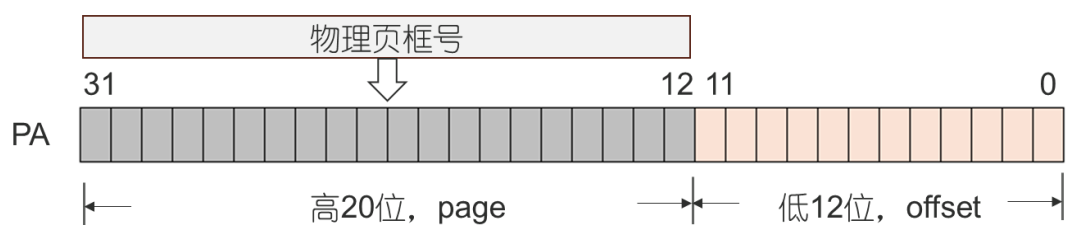
- 物理地址 [0, 0x1000), 0#物理页框
- 物理地址 [0x1000, 0x2000), 1#物理页框
-
- 物理地址 [n*4096, (n+1)*4096), n#物理页框
-

代码/数据的物理地址

现在让我们计算：进程 A，虚地址 VA 为 0x401008 的指令，物理地址是多少。已知，分配给逻辑页 0x401 的物理页框号是 0x500。解法非常简单。这条存放在 0x500#物理页框的指令，相较页框首地址 0x500000，偏移 8 个字节，所以，物理地址是 0x500008。不失一般性，

$$\text{物理地址 PA (Physical Address)} = \text{物理页框号 PageFrame} * 4096 + \text{offset}$$

MMU 不擅长乘法操作。这里，乘数是 2^n , $n=12$ ，求取物理地址 PA，MMU 的做法，用 20 位的物理页框号 PageFrame 盖掉 VA 的高 20 位，耗时 ≈ 0 。



进程的页表

内核为每个进程分配一张页表，登记分配给每个逻辑页的物理页框号。所以，页表是一个数组，下标是页号，数组元素是物理页框号。

例：进程 PA，代码段 2 页，数据段 2 页，用户栈 2 页。x_caddr = 0x500000, p_addr = 0x700000。求 PA 的页表。

解：

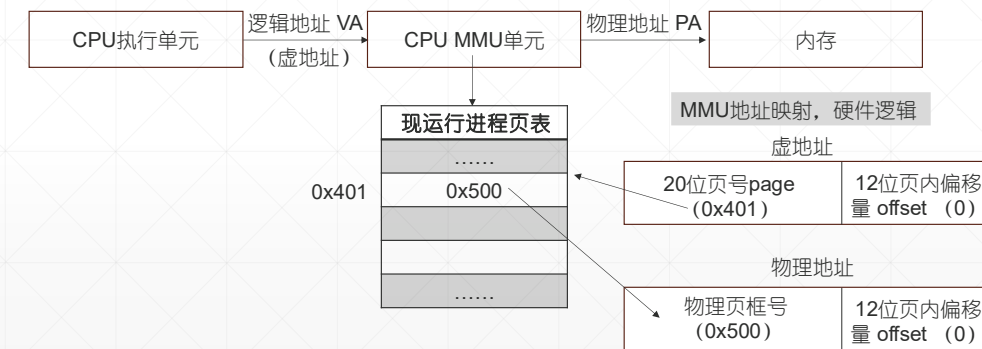
0	null	映射 用户 空间
1~0x400	 null	
0x401	0x500	
0x402	0x501	
0x403	0x701	
0x404	0x702	
0x405	null	
0x406~0x7fe	 null	
0x7ff	0x703	
	 null	
0xC0000	0	映射 核 心 空 间
0xC0001	1	
.....		
.....	0x3fe(1022)	
0xC03ff	0x700	
.....		

完整的地址映射过程



CPU增设内存管理单元 (Memory Management Unit)

MMU引用现运行进程的页表，将指令与数据的逻辑地址翻译成物理地址。



MMU 引用现运行进程的页表，把 CPU 执行单元送来的虚地址 VA 翻译成物理地址 PA，送物理内存。具体的硬件逻辑非常简单：

- (1) 取 VA 高 20 位，逻辑页号 page。
- (2) 以 page 为下标，查现运行进程页表，得到物理页框号 PageFrame[注 1]。
- (3) 生成的物理地址 PA，高 20 位是 PageFrame，低 12 位是 VA 的 offset[注 2]。

本例，CPU 执行单元取指 `push %eax`，给出虚地址 0x401000。MMU 用物理页框号 0x500 取代 VA 中的逻辑页号 0x401，得物理地址 PA，0x500000。

[注 1] CPU 内部设有一个页表寄存器，i386 是 CR3，它的值是现运行进程的页表的起始物理地址。页表中的元素叫做页表项 (Page Table Entry, 简称 PTE)，里面存有页框号 PageFrame。PTE，4 字节。地址映射时，目标 PTE 的地址 = CR3 + Page * 4 (页表是每个元素 4 字节的一维数组，这个公式是在计算下标 page 的数组元素的起始地址)。实际，硬件设计时，应该是 Page 左移 2 位，0#、1#bit 补 0 后与 CR3 相加。

[注 2] MMU 生成的物理地址，位数应该与物理总线的位数相等，不一定是 32 位。

[另] 内存变量才有地址，寄存器是没有地址的。

E01 作业 1：整理板书上的例题，再写一遍

作业 2：参考 PPT12，PPT17，画进程 PB 图像

